

BDA : Module 2

- Q.1) Secondary nameNode is backup of nameNode. True or false?
- ⇒ a) Secondary nameNode is not a backup of nameNode.
b) Its main job is to periodically merge NameNode's edit logs with File system Image.
c) It doesn't receive real time updates from NameNode.
d) In case NameNode crashes; secondary one can't immediately take over.
e) Manual recovery needed.

Q.2) Write map reduce pseudo-code for word count.
Apply map reduce working:
"This is NoSQL. NoSQL handles complex data".

⇒ 1) Preprocessing (lower, remove punctuation)
"this is nosql nosql handles complex data".

Map function:
function map (document_id, document) ↙ has statement
for each word in document.split():
emit(word.lower(), 1)

input:
map (1, " _____ " _____)

output :

("this", 1)
("is", 1)
("nasql", 1)
("nasql", 1)
("handles", 1)
("complex", 1)
("data", 1)

2) Shuffle & Sort [group values by word]

"this" → [1]
"is" → [1]
"nasql" → [1, 1]
"handles" → [1]
"complex" → [1]
"data" → [1]

3) Reduce function

↳ pass each word

function (word, values_list):

sum = 0

for value in values_list

sum += value

emit (word, sum)

O/P:

("this", 1)
("is", 1)
("nasql", 2)
("handles", 1) ("data", 1)
("complex", 1)

Q.3) Explain MapReduce pipeline with suitable example.

-
- a) MapReduce is a programming model used for processing large scale data in distributed environment.
 - b) 3 key phases:
 - 1) Map
 - 2) Shuffle & Sort
 - 3) Reduce
 - c) All these phases run in N^1 across Hadoop cluster.
 - d) Eg: Cat sat on the mat. Cat chased rat.

1) Preprocessing:
cat sat on the mat cat chased rat
split 1: —
each split sent to separate mapper.

Mapper 1: (O/P) Mapper 2:

("cat", 1)	("cat", 2)
("sat", 1)	("chased", 2)
("on", 1)	("rat", 2)
("the", 1)	
("mat", 1)	

2) Shuffle & Sort phase
[group values with same key, sort them.]

(O/P) :

("cat", [1, 1])

("sat", [1])

("on", [1])

("the", [1])

("mat", [1])

("chased", [1])

("rat", [1])

3) Reduce phase.

("cat", 2)

("sat", 1)

("on", 1)

("the", 1)

("mat", 1)

("chased", 1)

("rat", 1)

3.5) Selection Operation using MapReduce

Algorithm:

map (key, value):

for tuple in value:

if tuple satisfies condition:

emit (tuple, tuple)

reduce (key, values)

emit (key, key)

Map Worker 1				Map Worker 2			
A	B	A	B	A	B	A	B
1	2	1	2	2	3	1	1
3	1	3	5	4	5	2	1

Condition: $B \leq 2$

①

MW1

key	value
(1,2)	(1,2) (1,2)
(3,1)	(3,1)

MW2

key	value
(1,2)	(1,2)
(2,3)	(2,3)

A hash function applied:

②

MW1

key	value
(1,2)	(1,2) (1,2)
(3,1)	(3,1)

MW2

key	value
(1,1)	(1,1)
(2,1)	(2,1)

③

RW2

exchange

tables 2, 3 [to discard key redundancies]

k	v
(1,2)	(1,2)
(1,2)	(1,2)

k	v
(1,1)	(1,1)

k	v
(3,1)	(3,1)

k	v
(2,1)	(2,1)

④

RW2

(key, key)

k	v
(1,2)	(1,2) (1,2)
(1,1)	(1,1)

RW2

k	v
(3,1)	(3,1)
(2,1)	(2,1)

⑤

RW1

A	B
1	2
1	1

RW2

A	B
3	1
2	1

8.6) Projection using Map Reduce
 Let S be subset containing selected attributes
 (columns)

map(key, value):

for tuple in value:

ts = tuples with attributes in S .

emit(ts , ts)

reduce(key, values):

emit(key, key)

Qn) Project (B, C)

MW 1

A	B	C
1	2	3
3	1	4

A	B	C
2	2	3
4	3	4

MW 2

A	B	C
5	7	2
7	2	1

①

MW 1

k	v
(2,3)	(2,3) (2,3)
(1,4)	(1,4)
(3,4)	(3,4)

MW 2

k	v
(7,2)	(7,2)
(1,7)	(1,1)
(2,5)	(2,5)
(1,6)	(1,0)

A hash function will be applied:

② MW1 MW2

k	v
(2,3)	(2,3) (2,3)
(1,4)	(1,4)

k	v
(7,2)	(7,2) (7,2)
(1,1)	(1,1) (1,1)

③ RW1

k	v
(2,3)	(2,3) (2,3)
(1,4)	(1,4)

RW2

k	v
(3,4)	(3,4) (3,4)
(2,5)	(2,5) (2,5)
(1,6)	(1,6) (1,6)

④ RW1

k	v
(2,3)	(2,3) (2,3)
(1,4)	(1,4)
(7,2)	(7,2)
(1,1)	(1,1)

RW2

k	v
(3,4)	(3,4) (3,4)
(2,5)	(2,5) (2,5)
(1,6)	(1,6) (1,6)

only keys

⑤ RW1

B	C
2	3
1	4
4	2
1	1

RW2

B	C
3	4
2	5
1	6

Q.7) Union using Map Reduce

⇒ map (key, value):

for tuple in value:

emit (tuple, tuple)

reduce (key, values):

emit (key, key)

MW 1	
A/B	A/B
1/2	1/2
3/1	2/1

MW 2	
A/B	A/B
2/3	1/1
4/5	2/1

①

MW 1	
k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,1)	(2,1)

MW 2	
k	v
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,2)	(2,2)

apply hash function

②

MW 1	
k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,1)	(2,1)

MW 2	
k	v
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,2)	(2,2)

③ RW1 Discard key redundancy RW2

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,3)	(2,3)
(4,5)	(4,5)

k	v
(2,1)	(2,1)
(1,1)	(1,1)
(2,1)	(2,1)

④ RW1

k	v
(1,2)	(1,2)
(3,1)	(3,1)
(2,3)	(2,3)
(4,5)	(4,5)

RW2

k	v
(2,1)	(2,1)(2,1)
(1,1)	(1,1)

⑤

RW1

A	B
1	2
3	1
2	3
4	5

RW2

A	B
2	1
1	1

(key, key)

Q.8) Intersection using Map Reduce

map (key, value):
for tuple in value:
emit (tuple, tuple)

reduce (key, values):
if len(values) > 1:
emit (key, key)

mw1

A	B
1	2
3	1

A	B
1	2
2	1

mw2

A	B
2	3
4	5

A	B
1	1
2	1

①

mw1

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,1)	(2,1)

mw2

k	v
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,1)	(2,1)

apply hash function:

②

mw1

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,1)	(2,1)

mw2

k	v
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,1)	(2,1)

③

RW1

Discard key redundancies

RW2

k	v
(1,2)	(1,2) (1,2)
(3,1)	(3,1)

k	v
(2,3)	(2,3)
(4,5)	(4,5)

k	v
(1,2)	
(3,1)	
(2,1)	(2,1)

k	v
(1,1)	(1,1)
(2,1)	(2,1)

④

RW1

RW2

k	v
(1,2)	(1,2) (1,2)
(3,1)	(3,1)
(2,3)	(2,3)
(4,5)	(4,5)

k	v
(2,1)	(2,1) (2,1)
(1,1)	(1,1)

⑤ if $\text{len}(\text{values}) > 1$; then count

RW1

RW2

k	v
(1,2)	(1,2) (1,2)

k	v
(2,1)	(2,1) (2,1)

⑥

RW1

RW2

A	B
1	2

A	B
2	3

9) ~~map(key, value)~~ Difference using MapReduce

map(key, value):

if $key == R$:

for tuple in value:
emit(tuple, R)

else:

for tuple in value:
emit(tuple, S)

reduce(key, values)

if values == [R]:
emit(key, R)

Qⁿ)

A	B
1	2
3	1

MW 1 T2

A	B
1	2
2	1

T1 MW 2

A	B
2	3
4	5

A	B
1	1
2	1

①

MW 1

k	v
(1, 2)	[T2, T2]
(3, 1)	[T1]
(2, 1)	[T2]

MW 2

k	v
(2, 3)	[T1]
(4, 5)	[T1]
(1, 1)	[T2]
(2, 1)	[T2]

Apply hash function:

②

MW 2

k	v
(1, 2)	[T1, T2]
(3, 1)	[T1]
(2, 1)	[T2]

MW 2

k	v	k	v
(2, 3)	[T1]	(1, 1)	[T2]
(4, 5)	[T1]	(2, 1)	[T2]

③ Discard key redundancies

RW 1

k	v
(1,2)	[T1, T2]
(3,1)	[T4]
(2,3)	[T1]
(4,5)	[T4]

RW 2

k	v
(2,1)	[T2]
(1,1)	[T2]
(2,1)	[T2]

④

RW 1

k	v
(1,2)	[T1, T2]
(3,1)	[T1]
(2,3)	[T1]
(4,5)	[T1]

RW 2

k	v
(2,1)	[T2]
(1,1)	[T2]

Both
mw1 &
mw2
it was
in T2

eliminate keys both in T1 & T2;
only in T2

⑤

RW 1

k	v
(1,2)	[T1]
(3,1)	[T1]
(2,3)	[T1]
(4,5)	[T1]

RW 2

k	v
---	---

⑥

RW 1

A	B
3	2
2	3
4	5

RW 2

A	B
---	---